IIT POKERBOTS · 2026

Information Warfare Bot

DominatorBot

Team NOTATO · IIT Pokerbots 2026

Anant Jain & Sparsh Bandi

1141

Wins

316

Losses

2nd

Overall - Prelims

DominatorBot - Architecture Overview

PERCEIVE

Equity engine

O(1) preflop table lookup
Adaptive MC (90-650 sims)
Exhaustive river when opp card known

Board analyzer

Texture: paired · monotone · trips
17-class hand strength
OESD / gutshot / flush / combo draws

AUCTION

Sealed bid engine

Expected-Pot scaled bid sizing *(via linear regression)*

K-Means clustered equity tiers × randomized ranges

Empirical True Value modeling *(Offensive + Defensive values) tuned from bot logs*

ADAPT

Opponent model

18-hand rolling recalibration
65/35 recent vs cumulative blend
Derives: call_scale, aggro_boost

Time management

Scales sim budget to time bank
Degrades gracefully under pressure
20s total across 1000 rounds

EXECUTE

Decision engine

Street-specific skepticism
SPR + pot commitment gates
Position-aware preflop fold

Action selector

Check-raise with monsters
Draw semi-bluff sizing
Catch-bluff vs overbets

KEY INSIGHT

We won the auction

Exhaustive equity → near-perfect decisions
Massive value extraction (thin value bets)
Near-zero call margin → never fold the best hand

VS

They won the auction

Near-zero bluff frequency (they see through us)
Extra skepticism on opponent bets (+0.12 margin)
Tighter investment cap (max 25% stack w/o monster)

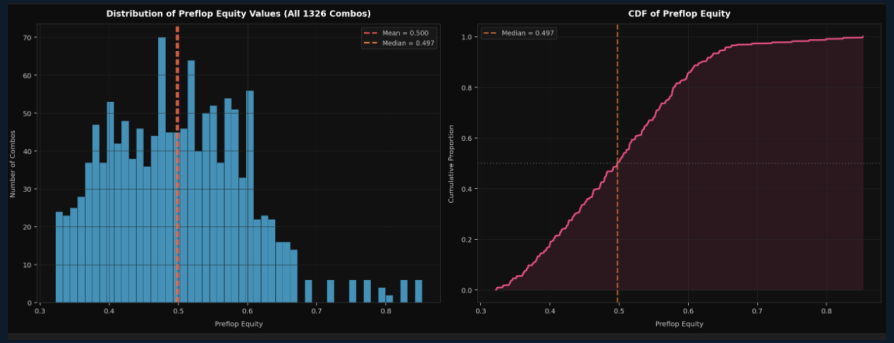
Equity Estimation - Monte Carlo, Exact Lookup & Exhaustive River

PRE-FLOP: Exact Table Lookup

- Full 1,326-entry table: every unique 2-card combo
- Indexed by canonical suit order (c < d < h < s)
- Eliminates all MC noise on most impactful street
- O(1) lookup - zero computation cost

POST-FLOP: Adaptive Monte Carlo

- Sim count scaled by street depth + time bank
- River + known opp card → exhaustive (~44 unknowns)
- Flop: 90–600 sims · Turn: 130–650 · River: 120–600
- Time bank checked every query - never runs over limit



Street	Opp Known?	Method
Preflop	N/A	Lookup Table
Flop/Turn	No / Yes	Monte Carlo
River	**Yes (1 card)**	**EXHAUSTIVE**

Key Edge: Exhaustive River Evaluation

On a complete river with one known opponent card, we enumerate all ~44 remaining deck cards for exact equity - no sampling error. This gives perfectly calibrated EV at the most decisive moment.

Our bot operates on a four-pillar architecture - Perceive, Auction, Adapt, and Execute - anchored heavily on whether we win or lose the information warfare auction.

Adjusted Equity - The Decision Engine

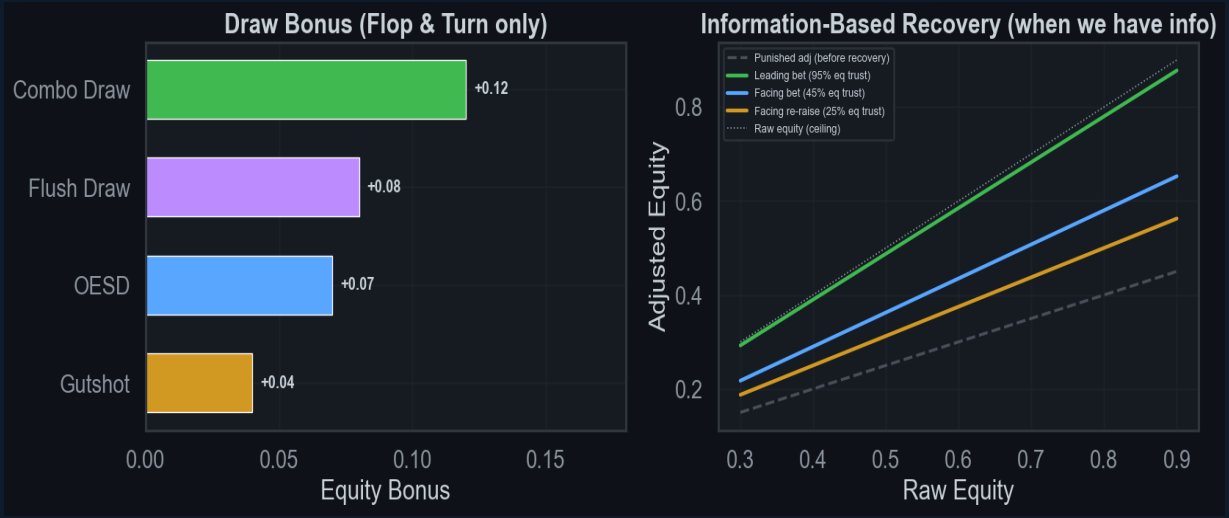
While raw equity calculates the pure mathematical probability of winning, adjusted equity determines the actual profitability of taking action.

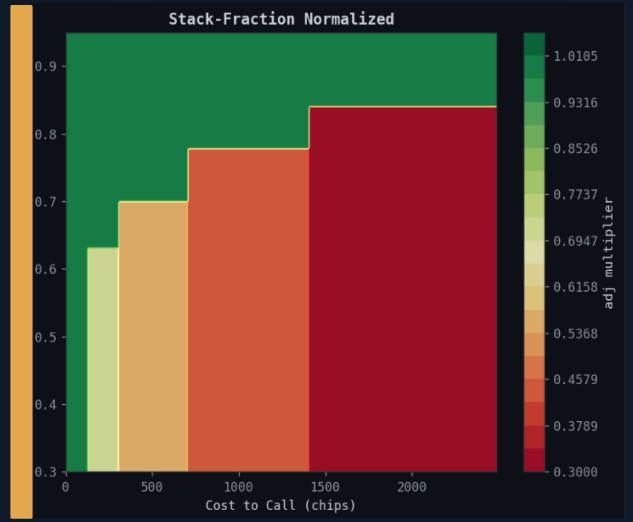
Raw Equity (eq) — MC / Table

Apply Penalties — bet, street, board, info

Apply Recovery — vs. aggro opp

Adjusted Equity (adj) — final metric





Left: equity bonus added for each draw type on flop/turn · Right: information-based recovery restoring adj toward raw eq

Confidence Punishment - Tiered Multiplier System

Our tiered confidence punishment system increasingly penalizes large bets the closer we get to the river, especially when we are at an information disadvantage

Hard Multipliers (Special Cases)

- River check-raise: ×0.30 (almost never a bluff)
- Turn overbet (br>1.0, eq<0.87): ×0.38
- Opp has info on us (br>0.20): ×0.60
- xp = paired +0.05, monotone +0.05, trips +0.10

Flop (Loosest)

Draws still live. ×0.32 - 0.84 range. Most forgiving penalties.

Turn (Medium)

One card to come. ×0.25 - 0.80 range. Tighter than flop.

River (Tightest)

No outs left. ×0.20 - 0.70 range. Only strong hands survive.

Information Asymmetry Penalty

When opponent holds info on us (br > 0.20, eq < 0.90): ×0.60 multiplier. Any meaningful bet from an info-holding opponent is treated as credible.

Confidence Recovery - Exploiting Aggressive Opponents

```
if self.call_scale < 1.0:
    rec = (eq - adj) × (1.0 - self.call_scale) × 0.55
    adj = min(eq, adj + rec)
```

call_scale < 1.0

Kicks in when opponent aggression crosses threshold. Higher aggression = lower scale.

Soft Cap at eq

adj never exceeds raw equity. This prevents over-folding without causing over-calling on weak hands.

Recovery Amount

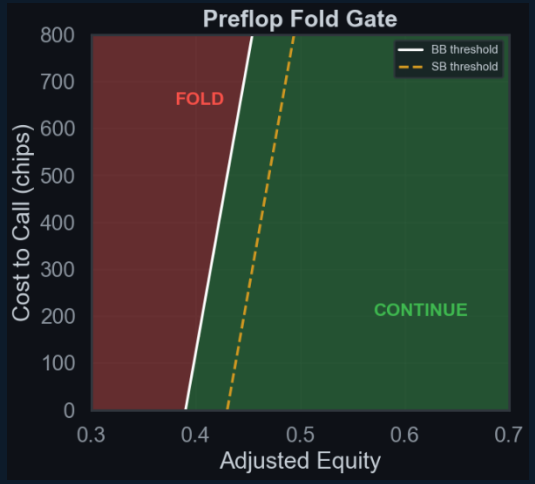
Proportional to the gap between adj and eq, scaled by (1 − call_scale). More aggro = bigger recovery.

Anti-Exploit Guard

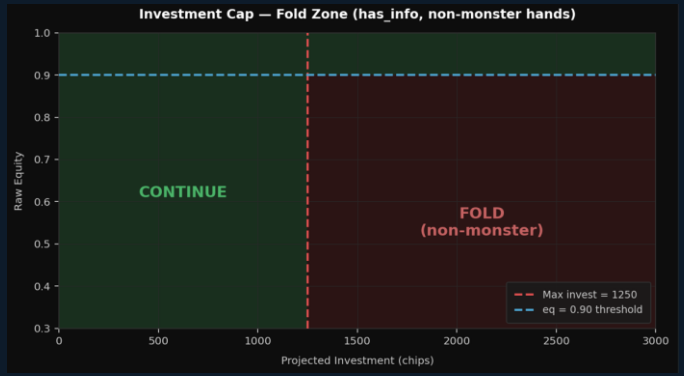
Prevents opponents from profitably bullying us with persistent aggression across an entire match.

Aggression Rate	call_scale	Effect
ba > 0.50 (hyper-aggro)	0.60	Strong recovery — rarely fold to large bets
0.40 < ba ≤ 0.50	0.70	Moderate recovery
0.30 < ba ≤ 0.40	0.82	Light recovery
ba < 0.12 (very passive)	1.28	Tight calling — fold more, let them bluff into us

Folding and Calling Strategy - Dynamic Equity Floors



Preflop



Post Flop

Condition	Equity Floor	Rationale
Pot > 700 or projected	54% (or pot odds, whichever higher)	Base commitment threshold in large pots
Already committed >22%	Just pot odds	Cheap to call
SPR < 1.5 and bet > 50%	< 38% (or pot odds)	Near stack-off = lower bar to continue
Already committed >55%	28% (or pot odds)	Almost all-in anyway, call
Paired board + large bet (>38%)	60% minimum	Paired boards inflate opponent trips

Pot commitment logic

Info Asymmetry Protection

- Opponent has our card: cap invest at 25% of stack
- Non-monster hand + proj_invest > 1250: FOLD
- Equity threshold for exception: eq ≥ 0.90
- BB fold gate lower than SB (already invested)
- Preflop gate: 0.39 + min(0.12, cr×0.40) (BB)
- Preflop gate: 0.43 + min(0.12, cr×0.40) (SB)

•If our bot has **extra information (see opponent's card)**, our equity is reliable → call as soon as it beats pot odds, no buffer needed.
•If **opponent has info on our bot**, our equity is likely overestimated → require a strong safety margin (~12%) before calling.

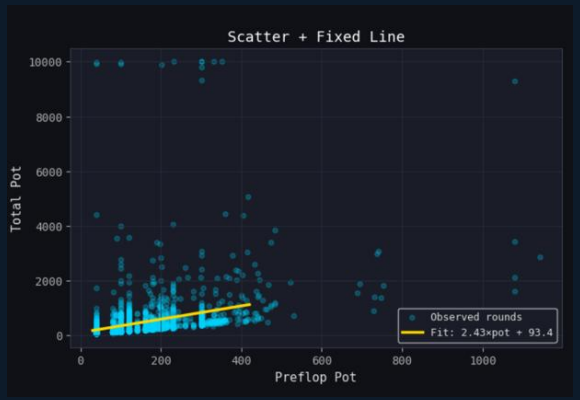
• Apply last bonus to decide whether call or fold on our adjusted equity and if

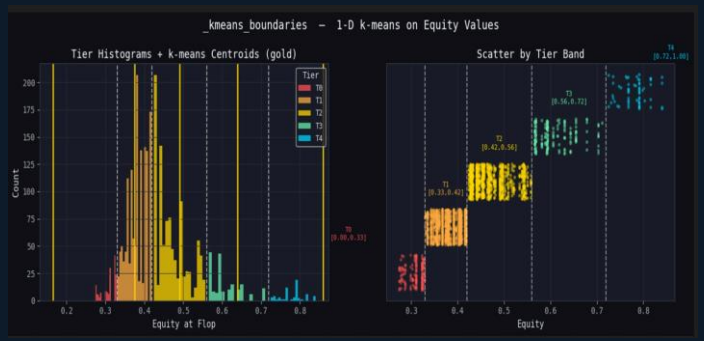
```
if adj > (p_odds + margin + pip):
        return ActionCall() if cs.can_act(ActionCall) else ActionCheck()
    else:
        return ActionFold() if cs.can_act(ActionFold) else ActionCheck()
```

Draw Type	Bonus
Combo (flush + straight)	+0.10
Flush draw	+0.07
Open-ended straight (OESD)	+0.06
Gutshot	+0.03
Draw Type	Bonus

Auction Optimizer - From Naive to Data-Driven

The auction is about gaining info AND denying it to the opponent.





Step 1: Expected Pot Estimation

```
total_pot = 2.43 × preflop_pot + 93.4
```

$R^2 = 0.674$ | Median |error| = 65 chips

Why this matters: A naive bid of 50 chips is ~26% of exp_pot when pot=40 but only ~5% when pot=400. Our bid must scale with expected pot to avoid systematic overbidding on small pots and underbidding on large ones. We approximated the fit to **exp_pot = 2.5 × pot + 100** for code simplicity.

Step 2: K-Means Equity Tier Clustering

T4 (Top)	T3	T2	T1	T0 (Low)
0.72	0.56–0.72	0.42–0.56	0.33–0.42	≤ 0.33

Why k-means? Uniform x-axis splits (e.g. 0.20 buckets) put most hands in 1-2 tiers. 1-D k-means clusters by density - placing tier boundaries where equity values naturally separate. This gives finer granularity in the crowded 0.35–0.55 zone where most flop equities land, and coarser bins at the extremes where hands are rare.

Step 3: True Value - Offensive + Defensive

Variable	Formula
Expected payoff	Δeq × exp_pot
off_base	mean(|Δeq| × exp_pot) over rounds where we won auction
emp_delta	trimmed_mean(payoff | we won) − trimmed_mean(payoff | opp won)
def_base	emp_delta − off_base
off_mult	12 · eq · (1 − eq) if eq ≤ 2/3; 4 · eq if eq > 2/3
def_weight	clip(0.70 + 1.20 · avg_eq, 0.70, 1.90)
off_tier	off_base × off_mult
def_tier	def_base × def_weight

Key insight: Off-base is the average chip value gained from auction information for a given equity tier, estimated via Monte Carlo (5000) simulations and scaled by expected pot size. Def-base is the defensive value of winning the auction, i.e., denying the opponent access to your card information.

Final Data Visualization

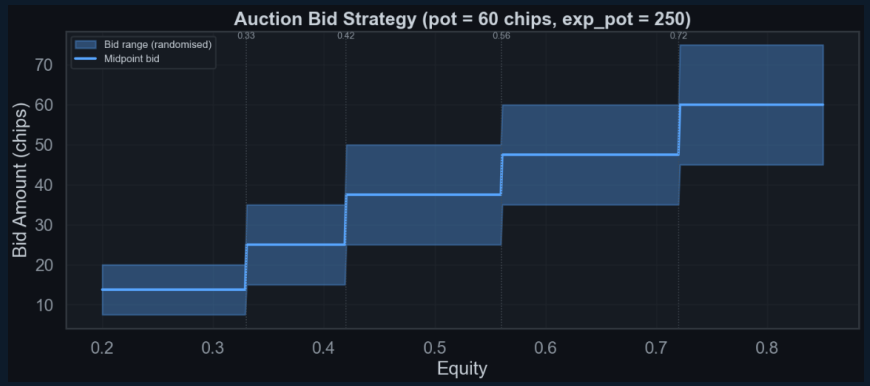
```
true_val
off_tier + def_tier
tier_opp = opp_bids[mask]

derive_bid_range()
bid_lo = true_val × lo_mult
bid_hi = true_val × hi_mult

fractions
frac_lo = bid_lo / avg_exp
frac_hi = bid_hi / avg_exp

blend w/ opponent
frac_lo = 0.55×theory + 0.45×opp_P60
frac_hi = 0.45×theory + 0.55×opp_P85

final bid (per tier)
bid = int(exp_pot × uniform(frac_lo, frac_hi))
```



Final Ranges and Equity Visualization

frac_lo (floor) & frac_hi (ceiling)

55% theory + 45% opponent q60 (60th percentile)
Lean on what the info is worth. Floor ensures we don't bid less than the expected value justifies.

45% theory + 55% opponent q85 (85th percentile)
Ceiling driven more by opponent aggression. If they bid aggressively, we must raise our max to compete.

```
exp_pot = pot × 2.5 + 100        # estimated total pot value

bid     = int(exp_pot × fraction)   # fraction ∈ [0.03, 0.30] scaled by equity
```

Raising Strategy - Street-by-Street

Pre-Flop

Condition	Action	Sizing
Facing 3-bet (cost > 600, adj > 0.80)	Raise	2.0–3.0× pot
adj > 0.80	Open raise	3.0–4.5× pot
adj > 0.64, cost ≤ 20	Medium raise	2.5–3.5× pot
adj > 0.52, cost ≤ 20	Small raise	2.0–3.0× pot
adj < 0.73, already	Skip raise	– (lean toward calling)

River

Situation	Condition	Action
Betting first	Has auction info + equity > 52%	Bet 28–40% pot
Betting first	Equity > 78%	Bet 55–90% pot
Betting first	Equity > 62%	Bet 40–65% pot
Betting first	Equity < 42%	Bluff 55–80% pot (rare)
Facing a bet	Equity > 88%, no info disadvantage	Re-raise 2.8–4.5× bet

Flop/Turn

Situation	Condition	Action
Facing a bet	Equity > 88%, monster hand	Check-raise 30% of the time (22%)
Facing a bet	Equity > 78%, info-reraise ok	Raise 2.8–4.0× bet (scaled down 15%)
Facing a bet	Has info + non-monster hand	No reraise call/fold only
Betting first	Equity > 96%	Slow-play check (4% of the time)
Betting first	Equity > 68%	Bet 70–125% pot, larger if strong

Raising Logic Summary

- Preflop: conservative with reraises, aggressive opens. Skip raise when adj < 0.73 and already invested.
- Flop & Turn: check-raise 22 - 30% with monsters. Semi-bluff draws 55 - 90% pot.
- River: value bet 55–90% when adj > 0.78. Precise 28–40% pot bets when we have opponent's card (info edge).
- Info guard: no reraise on flop/turn when info-advantaged, unless set / flush / straight with eq > 0.85.

Tactical Exploits - Bluff Catching & Slowplay

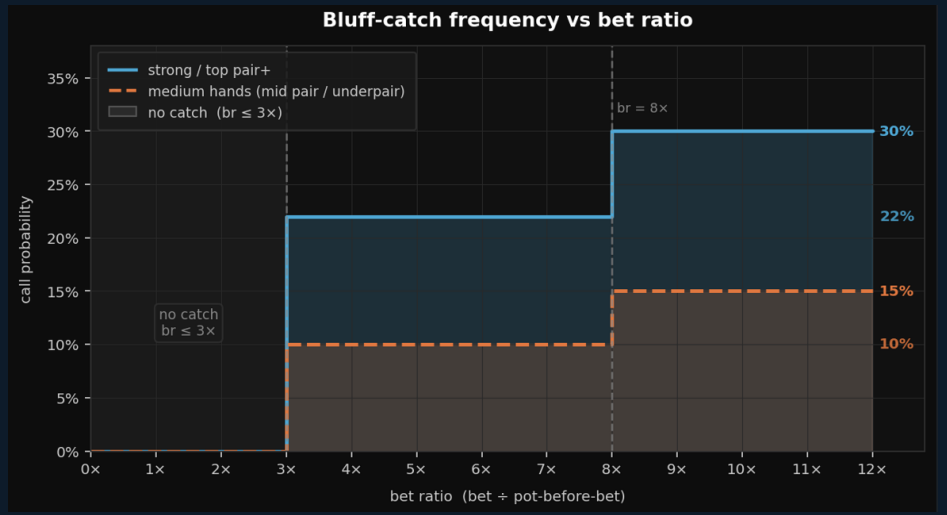
BLUFF CATCHING vs. OVERBETS

Overbets (>3× pot) are statistically polarised — nearly always either a bluff or the nuts.

Strong hands (set / two pair / flush / straight):
22% call freq → 30% for extreme bets (>8×)

Medium hands (mid pair / underpair):
10% call freq → 15% for extreme bets (>8×)

Randomisation prevents opponent from ever knowing if an overbet will be called — forces honest sizing.



SLOWPLAY TRAP (4% Check-Induction)

```
if adj > 0.96 and random.random() < 0.04:
    return ActionCheck()  # trap — induce bluff
```

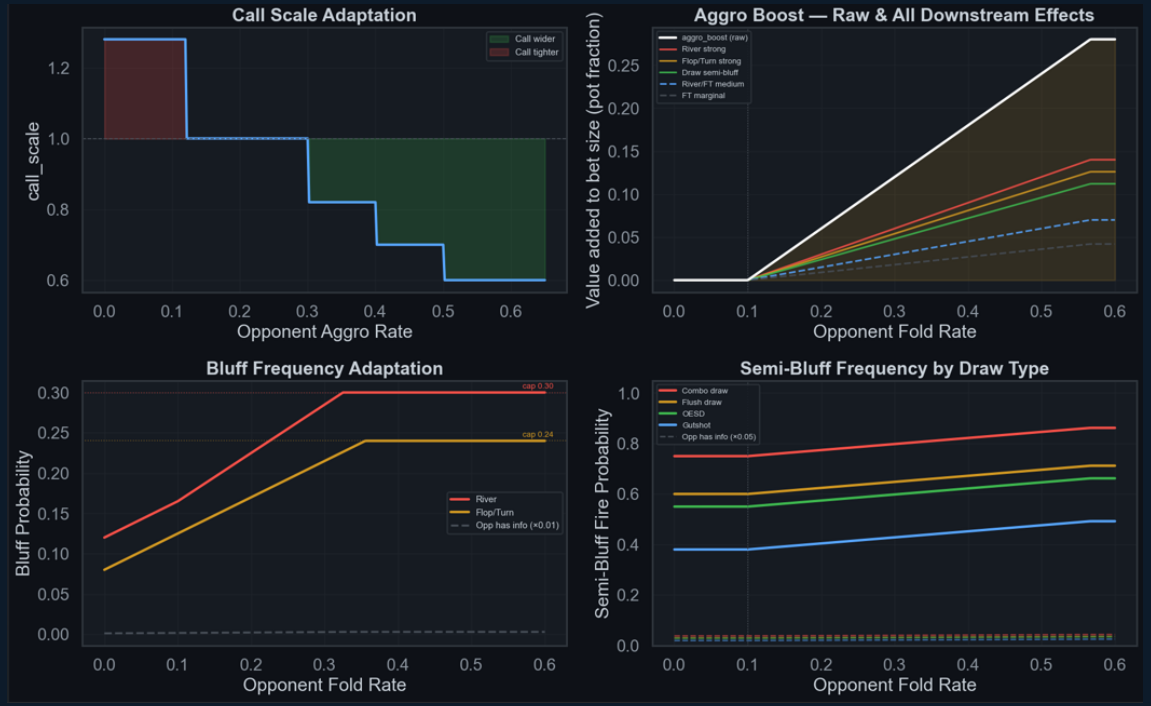
Near-nut hands check 4% of the time to bait opponent bluffs, then call or check-raise.

Info Edge Exploitation

Knowing opponent's card: river bet sizing shifts to precise 0.28–0.40× pot (thin value, not over-betting). Info-disadvantaged: raise frequency drops near zero, investment capped at 25% of starting stack for non-monster hands, as a failsafe to not overinvest with weak hands.

Opponent Modelling - Adaptive Strategy

Opponent stats are recalibrated every 18 hands using a 65% recent window / 35% cumulative blend, balancing short-term reads against long-term history to prevent overreacting to single-hand variance.



Knob	Function	Driven By	Effect
`call_scale`	Modulates call threshold	`opp_aggro_rate`	Aggro (>0.30) → tighter (0.60–0.82); passive (<0.12) → wider (1.28)
`aggro_boost`	Inflates bet sizing multipliers	`opp_fold_rate`	Activates at >0.10, caps at 0.28; added to sz across all streets
`bf`	Pure bluff fire probability	`opp_fold_rate` + `aggro_boost`	River `min(0.30, 0.12+…)`; Flop/Turn `min(0.24, 0.08+…)`; →×0.01 if opp has info
`sb`	Draw semi-bluff fire probability	`opp_fold_rate` + `aggro_boost` + draw type	Base 0.38–0.75 by draw; `+aggro_boost×0.40`; →×0.05 if opp has info
`opp_fold_rate`	Master observed stat	Opponent fold actions	Feeds `aggro_boost`, `bf`, and `sb` — high → bluff/semi-bluff more

Bot Randomization

Why Randomize?

The Exploitation Problem

Deterministic bots have fixed patterns. Advanced opponents model these patterns and develop perfect counter-strategies, turning every decision into a known quantity.

The Randomization Solution

By injecting calibrated randomness at every key decision point, DominatorBot creates a mixed strategy that no single deterministic counter can exploit. Even if an opponent models our ranges perfectly, the randomized frequencies make exploitation unprofitable.

Key Principle

Each randomization is weighted, not coin-flip random. Frequencies are tuned to equity thresholds, opponent tendencies, and empirical opponent modelling.

5 Randomization Trigger Points

Auction Bid — `Uniform Range`

Bids within equity-bucketed ranges using uniform randomization. Prevents opponents from reverse-engineering our hand strength from bid amounts.

Check-Raise — `22% – 30%`

Monsters are check-raised at balanced frequency. Prevents opponents from freely betting when we check.

Slow-Play Trap — `4% (adj > 0.96)`

Absolute monsters occasionally just check, inducing bluffs from opponents who misread passivity as weakness.

Overbet Catch — `10% – 30%`

Calls massive overbets at scaled probabilities. Prevents opponents from printing chips with unlimited overbet bluffs.

Pure Bluff — `8/12% – 24/30%`

Bluff frequency scales with opponent's historical fold rate. Maximises fold equity without over-bluffing.

Creates a highly exploit-resistant mixed strategy. Even if an opponent models our tendencies, calibrated randomness makes deterministic counter-attacks highly difficult

Bot Evolution - v1 → v2 → pl1 (v3) (Final Submission)

v1 → v2: Three Core Upgrades

MC Sim Budget
Before: Flat N=450/250/120
After: Adaptive N by street × info. River+info = exhaustive

Recal Window
Before: 30-hand window, slow
After: 18-hand, 65/35 blend - faster, noise-resistant

EQ Key Sort
Before: Alphabetical ('T'<'A')
After: Same bug - only fixed in pl1

v2 → pl1: Six Targeted Fixes

EQ Key Sort Fixed
Sort by (rank, suit) instead of alphabetical. All Ten combos now look up correctly.

Flush/Straight Classification
==4 for draws only (was ≥4). Made hands now get correct 1.25× sizing.

Catch-Bluff Logic Added
Calls overbets (>3× pot) at calibrated random freq: 22–30% for strong hands, 10–15% for medium. Prevents opponents printing chips with pure overbet bluffs.

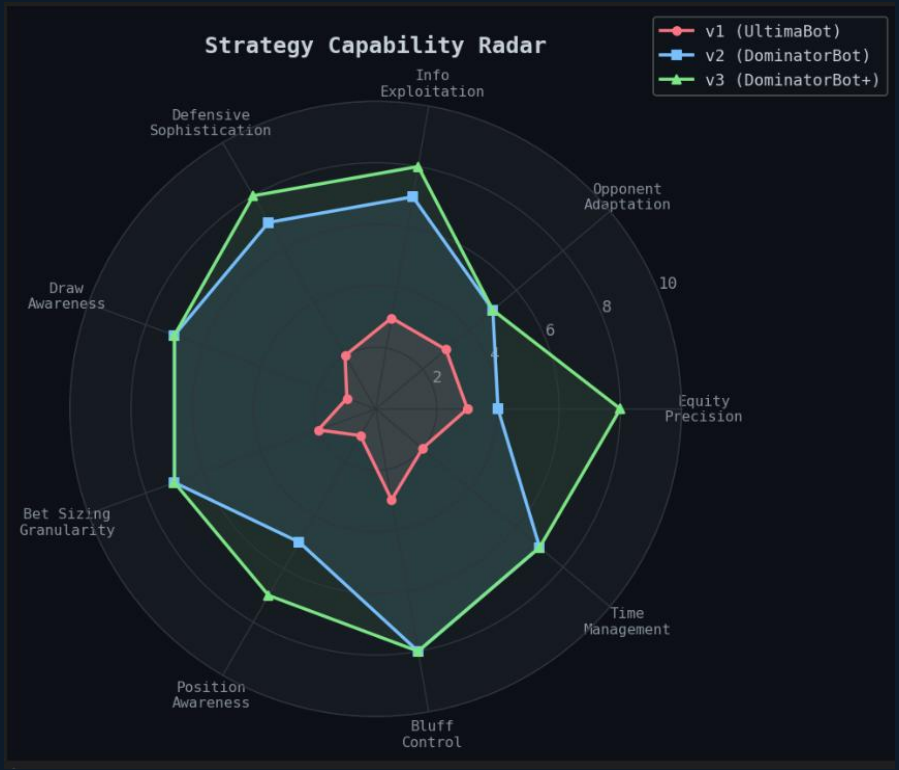
Auction Bid Reduction
18–30% of exp_pot (was 22–36%). Better aligned with actual info value.

Info Trust Blend
3-tier blend (was flat 0.45/0.55): facing re-raise: ×0.75/0.25; facing bet: ×0.55/0.45; no bet + info: ×0.05/0.95. More skepticism + 25% stack cap.

BB Gate Lowered
0.43 (SB/BB unified) → 0.39 BB / 0.43 SB split. Correctly defends hands like 8s7s that v1 folded. First introduced in v2.

```
# BUGGY (v2): key = ''.join(sorted(hand))  →  sorted(['Ts','Ac']) = 'AcTs' WRONG
# FIXED (pl1): sort by (_RV[rank], _SUIT_ORD[suit])  →  canonical lookup every time
```

Performance Analysis - Version Comparison

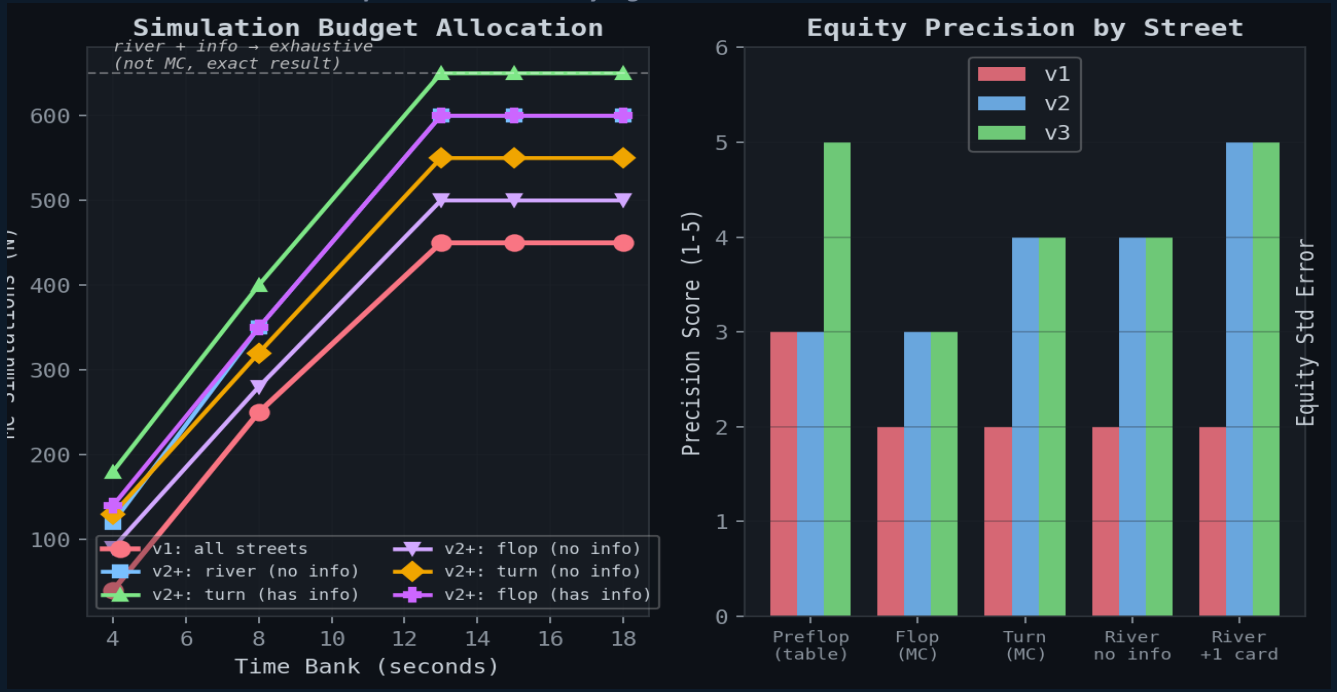


Strategy Capability Radar — Key Observations

- Equity Precision: sharpest spike (v2→v3), from EQ key fix + exhaustive river
- v1: near-perfect circle - uniformly weak on every axis
- Position Awareness: floor across all versions. The one unsolved dimension.
- Defense + Info Exploitation: grow in lockstep - same auction-state system drives both
- v3: right-heavy polygon. Strong at reading situations, shallow on positional execution.
- v2: asymmetric middle. Concave on Bluff Control and Bet Sizing - exactly where v3's fixes landed.

Our strategy capability radar visualizes the massive leap in equity precision and information exploitation from our initial build to the final competitive version.

By intelligently reallocating our simulation budget, particularly with exhaustive river enumerations, we drastically improved our equity precision while staying within our strict time bank.



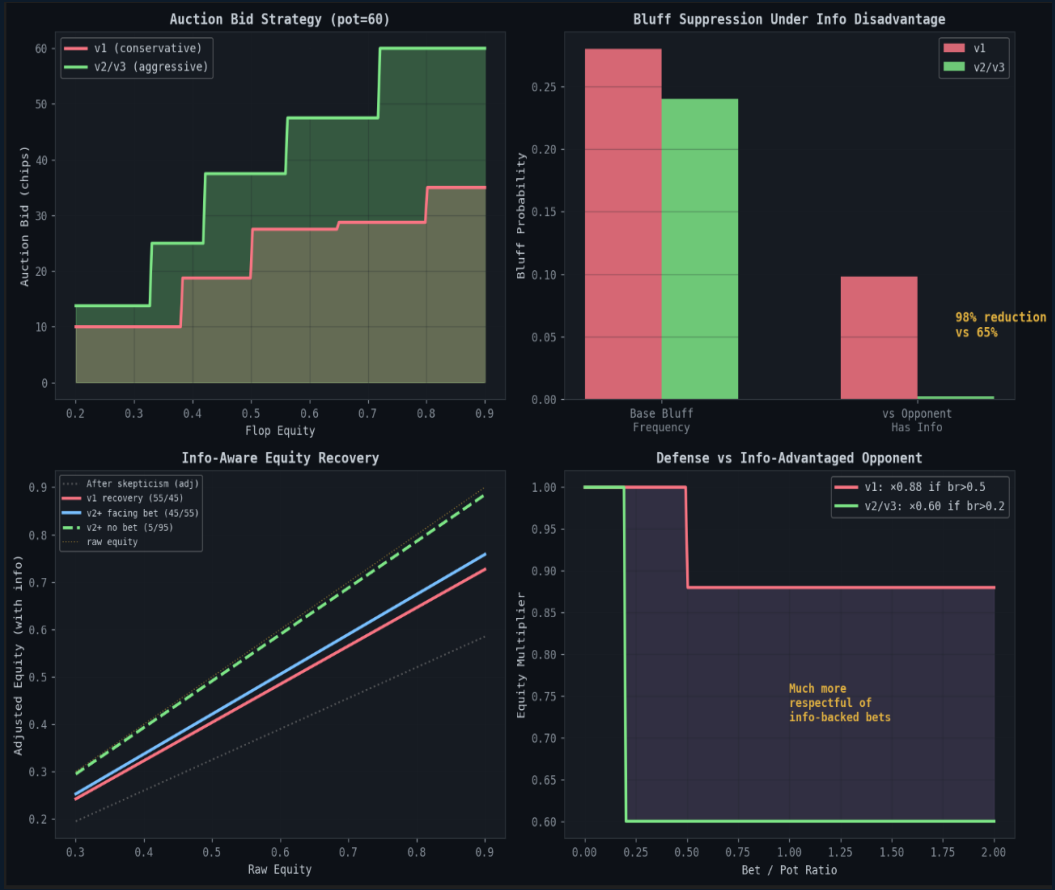
Simulation Budget

v3 pushes up to 650 sims on info-rich streets. On river with known opp card, drops MC entirely for exhaustive enumeration.

Equity Precision by Street

v3's biggest gain is preflop (table lookup fix) and river+1 card (exhaustive mode) — the two endpoints. Flop and turn remain shared ground between v2 and v3.

Comparing our iterations shows how the final bot significantly suppresses bluffs under an information disadvantage while bidding much more aggressively during the auction.



Auction Bid Strategy

v2/v3 bids nearly double v1 at every equity level. Info value scales with hand strength, and second-price means overbidding is the only real cost.

Bluff Suppression Under Info

When opponent holds info, v2/v3 cuts bluffing by 98% vs v1's 65%. Near-total suppression when bluffs are likeliest to get caught.

Info-Aware Equity Recovery

Holding info + no bet: adj ≈ raw equity (5/95 blend). Facing a bet: pulls back to 55/45, correctly discounting opponent signal.

Defense vs Info Opponent

v1: ×0.88 penalty only after br > 0.5 too little, too late. v2/v3: ×0.60 from br > 0.2, treating any bet from info-holder as credible.

Tournament.py - Local Bot Evaluation Engine

Custom round-robin framework for head-to-head testing all bot variants before submission.

Round-Robin Pairings

Every bot plays every other (nC2 × best-of-N). Results ranked by match wins, then bankroll.

```
ranking = sorted(stats.items(), key=lambda x: (x[1]["match_wins"],
x[1]["total_bankroll"]), reverse=True)  # sort by wins, then bankroll (desc)
```

How It Drove Improvement

We iteratively developed multiple bot variants, isolating features like SPR awareness, aggression thresholds, and auction strategy. By analyzing tournament match logs, we identified the most synergistic feature combinations and continuously re-ran tournaments to refine our core model.

Parallel Workers + Cache

ProcessPoolExecutor for parallel matches. JSON cache stores past results — reruns only compute new matchups.

```
# Workers init ONCE (not per-match).  Cache prevents replaying known
results.
cached = _try_resolve_from_cache(a, b, key, cache, retries)  # instant if
found
```

How It Drove Improvement

This approach significantly reduced processing time. If there are n bots that have already played $\binom{n}{2} \cdot m$ matches, then when a new bot is added, only $\binom{n+1}{2} \cdot m - \binom{n}{2} \cdot m$ additional matches need to be run.

Draw Replay Until Decisive

Tied matches replayed automatically. - best-of flag runs 3×/5×/10× reps to separate real edges from variance.

How It Drove Improvement

Ensured every code change was tested against the full opponent field before uploading. In total, we developed around 120 bots using this approach and selected the one with the best performance, minimal overfitting, and lowest variance.

Demo Tournament Output

Rank	Bot	W	L	D	Played	Total Bankroll	Avg Bankroll	Draw Replays
1	AlphaBot	2	1	0	3	+270	+90.0	0
2	GammaBot	2	1	0	3	+100	+33.3	0
3	BetaBot	1	2	0	3	-100	-33.3	0
4	DeltaBot	1	2	0	3	-270	-90.0	0

Missed Opportunities

Design choices that were directionally right but couldn't be implemented due to time constraints/ proper testing.

MC Redundancy — No Street Cache

✗ Waste: The same board produces identical equity, so caching results per street would have reduced redundant computation and freed up CPU resources for higher-N simulations where they matter most. We could have saved up to 500k simulations and instead run $\binom{45}{2}$ exhaustive simulations for the turn (with has_info) and river (without has_info) to obtain more precise equity estimates

```
# Current: _equity() called fresh on EVERY get_move() call
# Fix sketch:
if self._cached_street != street or self._cached_board != board:
    self._cached_eq = self._run_mc(cs, gi)
    self._cached_street, self._cached_board = street, board
     eq = self._cached_eq
```

Adaptive Auction Strategy Based on Opponent Bidding

We have dead code in our bot for opponent bid history that is never used, although it was intended to be part of our updated auction logic. After winning the auction, large raises can signal our information edge and cause opponents to fold. Randomizing between information-sized and standard bets would help disguise this tell. We were testing a much better auction strategy that uses opponent bid history, but we couldn't integrate it into our main codebase due to time constraints.

```
if my_cost > 0 and has_info:
   self.opp_bid_history.append(my_cost) # append known opponent bidding
elif my_cost == 0 and self.opp_has_info:
   est = max(self.my_bid + 5, self.my_bid * 1.5) # estimate opponent
bidding to be close to ours if we lose and append
if self.opp_bid_history:
   recent = self.opp_bid_history[-80:]
   self.opp_bid_avg = sum(recent) / len(recent)
```

```
info_curve = 4.0 * eq * (1.0 - eq) # peaks at eq=0.5
deny_bonus = max(0.0, (eq - 0.55) * 0.12) # strong hands deny info
base_bid = int(exp_pot * (info_curve * 0.14 + deny_bonus)
if eq < 0.32: # weak hand — info worthless
   base_bid = min(base_bid, max(0, int(exp_pot * 0.02)+ 20))
if len(self.opp_bid_history) >= 8:
   cap = int(self.opp_bid_avg * 2.5 + 15)
   base_bid = min(base_bid, cap) # don't overbid a frugal opp
bid = max(0, min(base_bid, my_chips)) self.my_bid = bid return
ActionBid(bid)
```

Key Advantages and Takeaways

IIT Pokerbots 2026 · Preliminary Round

DominatorBot excels through precise hand reading, aggressive information control, and a robust architecture that adapts quickly while remaining stable under pressure.

ADVANTAGES

Precision equity

1,326 suit-aware combos

O(1) preflop table → adaptive MC (90–650 sims) → exhaustive river enum when opp card known

Auction dominance

2nd 2nd-price advantage

K-Means clustered tiers with opponent-aware ranges; win = capture True Value (Offense + Defense), lose = force opponent to overpay.

Dual play modes

2 asymmetric modes

Won auction: thin value bets, near-zero call margin.
Lost auction: zero bluffs, 25% stack cap

Deep hand reading

17 hand classes

Full classifier (SF → set → TPGK → draws → nothing) + board texture + street-specific skepticism

Fast adaptation

18 hand recalibration

65/35 recent-vs-cumulative blend → derives call_scale + aggro_boost; degrades gracefully on time

Exploit resistance

5 defense layers

Position-aware fold gate, SPR commitment logic, check-raise traps (30%), catch-bluff calls (22–30%)

TAKEAWAYS

Information > chips

The auction is the real game. A 2nd-price bid that wins cheaply gives exact equity - turning every subsequent street into a near-solved decision.

Asymmetry is alpha

Most bots play identically regardless of who sees the card. Separate strategies for "we have info" vs "they have info" extracts value others leave on the table.

Adapt fast, degrade gracefully

With 20s across 1,000 rounds, every millisecond counts. A 3-tier equity engine (table → MC → exhaustive) gives the right precision while staying under budget.